

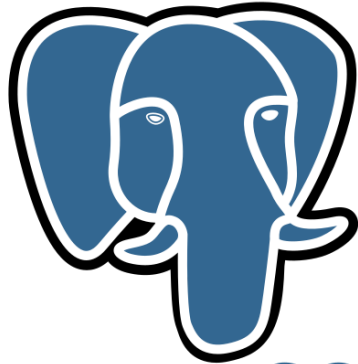
# Evaluación del impacto de los Modelos de Lenguaje en la Optimización de Consultas SQL para Diferentes Motores de Procesamiento de Datos

Gabriel Cárdena López



UNIVERSIDAD  
NEBRIJA

# Motores de Procesamiento de Datos



PostgreSQL



LLM's



**GPT-5**



Claude Sonnet 4.5



# Query's Base

## Query 1



```
1  SELECT l_returnflag,
2         l_linestatus,
3         COUNT(*) as count_order,
4         SUM(l_quantity) as sum_qty,
5         AVG(l_quantity) as avg_qty
6  FROM lineitem
7  WHERE l_shipdate ≤ DATE '1998-12-01'
8  GROUP BY l_returnflag, l_linestatus
9  ORDER BY l_returnflag, l_linestatus
```

## Query 2



```
1  SELECT
2      c.c_mktsegment,
3      COUNT(DISTINCT c.c_custkey) AS total_clientes,
4      AVG(c.c_acctbal) AS balance_promedio,
5      SUM(l.l_extendedprice * (1 - l.l_discount)) AS facturacion_total
6  FROM customer c
7  JOIN orders o ON c.c_custkey = o.o_custkey
8  JOIN lineitem l ON o.o_orderkey = l.l_orderkey
9  WHERE c.c_acctbal > 5000
10     AND o.o_orderdate BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'
11     AND l.l_returnflag = 'N'
12     AND c.c_custkey IN (
13         SELECT c2.c_custkey
14         FROM customer c2
15         WHERE c2.c_acctbal > (
16             SELECT AVG(c3.c_acctbal)
17             FROM customer c3
18         )
19     )
20  GROUP BY c.c_mktsegment
21  HAVING COUNT(DISTINCT o.o_orderkey) > 10
22  ORDER BY facturacion_total DESC
```



# Query 1 PostgreSQL

GPT- 5



```
1 SELECT
2     l_returnflag,
3     l_linestatus,
4     COUNT(*) AS count_order,
5     SUM(l_quantity) AS sum_qty,
6     AVG(l_quantity) AS avg_qty
7 FROM lineitem
8 WHERE l_shipdate < DATE '1998-12-02'
9 GROUP BY 1, 2
10 ORDER BY 1, 2
```

Gemini 2.5



```
1 SELECT l_returnflag,
2         l_linestatus,
3         COUNT(*) as count_order,
4         SUM(l_quantity) as sum_qty,
5         AVG(l_quantity) as avg_qty
6 FROM lineitem
7 WHERE l_shipdate ≤ DATE '1998-12-01'
8 GROUP BY l_returnflag, l_linestatus
9 ORDER BY l_returnflag, l_linestatus
```

Claude 4.5



```
1 SELECT
2     l_returnflag,
3     l_linestatus,
4     COUNT(*) as count_order,
5     SUM(l_quantity) as sum_qty,
6     AVG(l_quantity) as avg_qty
7 FROM lineitem
8 WHERE l_shipdate ≤ DATE '1998-12-01'
9 GROUP BY l_returnflag, l_linestatus
10 ORDER BY l_returnflag, l_linestatus
```



# Query 1 SPARK

## GPT- 5

```
1 WITH lineitem_filtered AS (  
2     SELECT  
3         l_returnflag,  
4         l_linestatus,  
5         l_quantity  
6     FROM lineitem  
7     WHERE l_shipdate ≤ DATE '1998-12-01'  
8 )  
9 SELECT  
10     l_returnflag,  
11     l_linestatus,  
12     COUNT(*) AS count_order,  
13     SUM(l_quantity) AS sum_qty,  
14     AVG(l_quantity) AS avg_qty  
15 FROM lineitem_filtered  
16 GROUP BY l_returnflag, l_linestatus  
17 ORDER BY l_returnflag, l_linestatus
```

## Gemini 2.5

● ● ●

```
1 ELECT /*+ COALESCE(3) */  
2     l_returnflag,  
3     l_linestatus,  
4     COUNT(*) as count_order,  
5     SUM(l_quantity) as sum_qty,  
6     AVG(l_quantity) as avg_qty  
7 FROM lineitem  
8 WHERE l_shipdate ≤ DATE '1998-12-01'  
9 GROUP BY l_returnflag, l_linestatus  
10 ORDER BY l_returnflag, l_linestatus
```

## Claude 4.5

● ● ●

```
1 SELECT /*+ REPARTITION(l_returnflag, l_linestatus) */  
2     l_returnflag,  
3     l_linestatus,  
4     COUNT(1) as count_order,  
5     SUM(l_quantity) as sum_qty,  
6     SUM(l_quantity) / COUNT(1) as avg_qty  
7 FROM lineitem  
8 WHERE l_shipdate < date'1998-12-02'  
9 GROUP BY l_returnflag, l_linestatus  
10 ORDER BY l_returnflag, l_linestatus
```



# Query 2 PostgreSQL

GPT - 5

```
1 SELECT AVG(c_acctbal) AS avg_bal
2   FROM customer
3 ),
4 FilteredCustomers AS (
5   SELECT
6     c.c_custkey,
7     c.c_mktsegment,
8     c.c_acctbal
9   FROM customer c
10  JOIN CustomerAvgBalance cab ON c.c_acctbal > cab.avg_bal
11  WHERE c.c_acctbal > 5000
12 ),
13 GroupedData AS (
14   SELECT
15     fc.c_mktsegment,
16     COUNT(DISTINCT fc.c_custkey) AS total_clientes,
17     AVG(fc.c_acctbal) AS balance_promedio,
18     COUNT(DISTINCT o.o_orderkey) AS distinct_orders_count,
19     SUM(l.l_extendedprice * (1 - l.l_discount)) AS facturacion_total
20   FROM FilteredCustomers fc
21  JOIN orders o ON fc.c_custkey = o.o_custkey
22  JOIN lineitem l ON o.o_orderkey = l.l_orderkey
23  WHERE o.o_orderdate BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'
24  AND l.l_returnflag = 'N'
25  GROUP BY fc.c_mktsegment
26 )
27 SELECT
28   c_mktsegment,
29   total_clientes,
30   balance_promedio,
31   facturacion_total
32 FROM GroupedData
33 WHERE distinct_orders_count > 10
34 ORDER BY facturacion_total DESC
```

Gemini 2.5

```
1 WITH CustomerAvgBalance AS (
2   SELECT AVG(c_acctbal) AS avg_bal
3   FROM customer
4 ),
5 FilteredCustomers AS (
6   SELECT
7     c.c_custkey,
8     c.c_mktsegment,
9     c.c_acctbal
10  FROM customer c
11  JOIN CustomerAvgBalance cab ON c.c_acctbal > cab.avg_bal
12  WHERE c.c_acctbal > 5000
13 ),
14 GroupedData AS (
15   SELECT
16     fc.c_mktsegment,
17     COUNT(DISTINCT fc.c_custkey) AS total_clientes,
18     AVG(fc.c_acctbal) AS balance_promedio,
19     COUNT(DISTINCT o.o_orderkey) AS distinct_orders_count,
20     SUM(l.l_extendedprice * (1 - l.l_discount)) AS facturacion_total
21   FROM FilteredCustomers fc
22  JOIN orders o ON fc.c_custkey = o.o_custkey
23  JOIN lineitem l ON o.o_orderkey = l.l_orderkey
24  WHERE o.o_orderdate BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'
25  AND l.l_returnflag = 'N'
26  GROUP BY fc.c_mktsegment
27 )
28 SELECT
29   c_mktsegment,
30   total_clientes,
31   balance_promedio,
32   facturacion_total
33 FROM GroupedData
34 WHERE distinct_orders_count > 10
35 ORDER BY facturacion_total DESC
```

Claude 4.5

```
1 WITH avg_balance AS MATERIALIZED (
2   SELECT AVG(c_acctbal) as avg_bal
3   FROM customer
4   WHERE c_acctbal > 5000
5 ),
6 qualified_customers AS MATERIALIZED (
7   SELECT
8     c.c_custkey,
9     c.c_mktsegment,
10    c.c_acctbal
11  FROM customer c
12  CROSS JOIN avg_balance ab
13  WHERE c.c_acctbal > ab.avg_bal
14        AND c.c_acctbal > 5000
15 ),
16 order_revenue AS (
17   SELECT
18     o.o_custkey,
19     o.o_orderkey,
20     SUM(l.l_extendedprice * (1 - l.l_discount)) as revenue
21  FROM orders o
22  INNER JOIN lineitem l ON o.o_orderkey = l.l_orderkey
23  WHERE o.o_orderdate ≥ '1995-01-01'::date
24        AND o.o_orderdate < '1997-01-01'::date
25        AND l.l_returnflag = 'N'
26  GROUP BY o.o_custkey, o.o_orderkey
27 )
28 SELECT
29   qc.c_mktsegment,
30   COUNT(DISTINCT qc.c_custkey) as total_clientes,
31   AVG(qc.c_acctbal) as balance_promedio,
32   SUM(orr.revenue) as facturacion_total
33 FROM qualified_customers qc
34  INNER JOIN order_revenue orv ON qc.c_custkey = orv.o_custkey
35  GROUP BY qc.c_mktsegment
36  HAVING COUNT(DISTINCT orv.o_orderkey) > 10
37  ORDER BY facturacion_total DESC
```



# Query 2 SPARK

GPT - 5

```
1 WITH global_avg AS (  
2   SELECT AVG(c.acctbal) AS avg_acctbal  
3   FROM customer  
4 ),  
5 qualified_customers AS (  
6   SELECT  
7     c_custkey,  
8     c_mktsegment,  
9     c_acctbal  
10  FROM customer c  
11  CROSS JOIN global_avg g  
12  WHERE c.c_acctbal > 5000  
13        AND c.c_acctbal > g.avg_acctbal  
14 ),  
15 filtered_orders AS (  
16   SELECT  
17     o_orderkey,  
18     o_custkey,  
19     o_orderdate  
20  FROM orders  
21  WHERE o_orderdate ≥ DATE '1995-01-01'  
22        AND o_orderdate < DATE '1997-01-01'  
23 ),  
24 filtered_lineitem AS (  
25   SELECT  
26     l_orderkey,  
27     l_extendedprice,  
28     l_discount  
29  FROM lineitem  
30  WHERE l_returnflag = 'N'  
31 )  
32 SELECT  
33   qc.c_mktsegment,  
34   COUNT(DISTINCT qc.c_custkey) AS total_clientes,  
35   AVG(qc.c_acctbal) AS balance_promedio,  
36   SUM(fl.l_extendedprice * (1 - fl.l_discount)) AS facturacion_total  
37 FROM qualified_customers qc  
38 JOIN filtered_orders fo ON qc.c_custkey = fo.o_custkey  
39 JOIN filtered_lineitem fl ON fo.o_orderkey = fl.l_orderkey  
40 GROUP BY qc.c_mktsegment  
41 HAVING COUNT(DISTINCT fo.o_orderkey) > 10  
42 ORDER BY facturacion_total DESC
```

Gemini 2.5

```
1 SELECT /*+ REPARTITION(c_mktsegment), BROADCAST(cab) */  
2   c.c_mktsegment,  
3   COUNT(DISTINCT c.c_custkey) AS total_clientes,  
4   AVG(c.c_acctbal) AS balance_promedio,  
5   SUM(l.l_extendedprice * (1 - l.l_discount)) AS facturacion_total  
6 FROM customer c  
7 JOIN (  
8   SELECT AVG(c.acctbal) AS avg_bal FROM customer  
9 ) cab ON c.c_acctbal > cab.avg_bal  
10 JOIN orders o ON c.c_custkey = o.o_custkey  
11 JOIN lineitem l ON o.o_orderkey = l.l_orderkey  
12 WHERE c.c_acctbal > 5000  
13        AND o.o_orderdate BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'  
14        AND l.l_returnflag = 'N'  
15 GROUP BY c.c_mktsegment  
16 HAVING COUNT(DISTINCT o.o_orderkey) > 10  
17 ORDER BY facturacion_total DESC
```

Claude 4.5

```
1 WITH avg_balance AS (  
2   SELECT /*+ BROADCAST(customer) */ AVG(c.acctbal) as avg_bal  
3   FROM customer  
4   WHERE c.acctbal > 5000  
5 ),  
6 qualified_customers AS (  
7   SELECT /*+ BROADCAST(ab) */  
8     c.c_custkey,  
9     c.c_mktsegment,  
10    c.c_acctbal  
11  FROM customer c  
12  CROSS JOIN avg_balance ab  
13  WHERE c.c_acctbal > ab.avg_bal  
14        AND c.c_acctbal > 5000  
15 ),  
16 filtered_data AS (  
17   SELECT  
18     o.o_custkey,  
19     o.o_orderkey,  
20     l.l_extendedprice * (1 - l.l_discount) as item_revenue  
21  FROM orders o  
22  INNER JOIN lineitem l ON o.o_orderkey = l.l_orderkey  
23  WHERE o.o_orderdate ≥ date'1995-01-01'  
24        AND o.o_orderdate < date'1997-01-01'  
25        AND l.l_returnflag = 'N'  
26 ),  
27 customer_orders AS (  
28   SELECT /*+ BROADCAST(qc) */  
29     qc.c_mktsegment,  
30     qc.c_custkey,  
31     qc.c_acctbal,  
32     fd.o_orderkey,  
33     fd.item_revenue  
34  FROM qualified_customers qc  
35  INNER JOIN filtered_data fd ON qc.c_custkey = fd.o_custkey  
36 )  
37 SELECT  
38   c_mktsegment,  
39   COUNT(DISTINCT c_custkey) as total_clientes,  
40   AVG(c_acctbal) as balance_promedio,  
41   SUM(item_revenue) as facturacion_total  
42 FROM customer_orders  
43 GROUP BY c_mktsegment  
44 HAVING COUNT(DISTINCT o_orderkey) > 10  
45 ORDER BY facturacion_total DESC
```



# Resultados

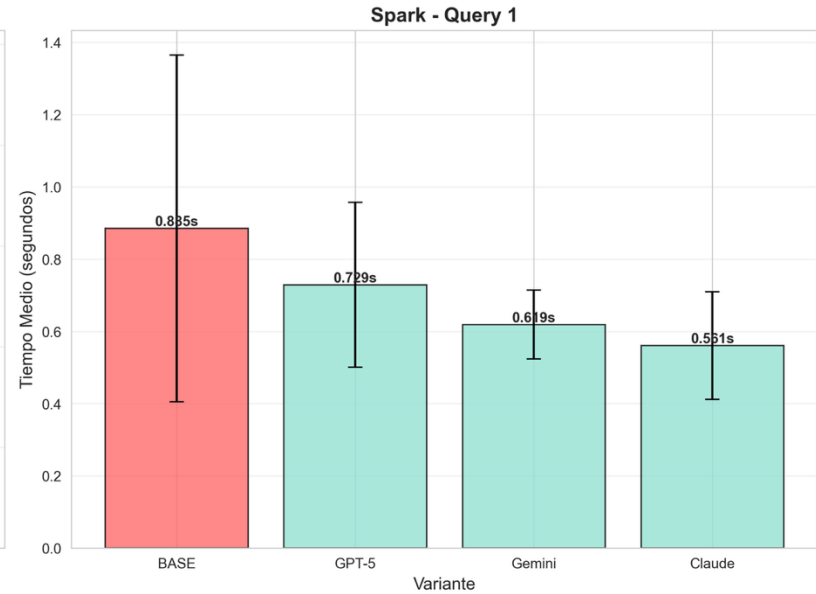
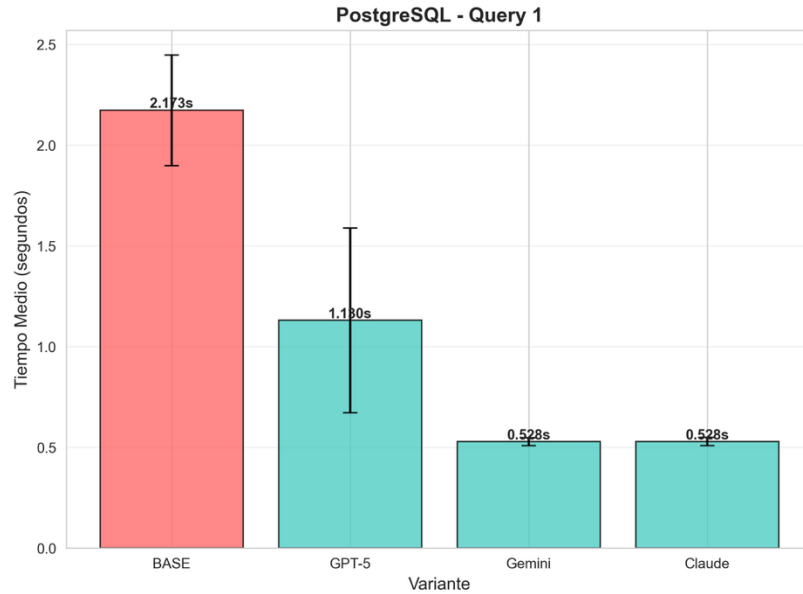
Estadísticas Completas del Benchmark

| Query                     | Motor    | Media (s) | Std Dev (s) | Min (s) | Max (s) | Mediana (s) | Iteraciones |
|---------------------------|----------|-----------|-------------|---------|---------|-------------|-------------|
| Query 1 (BASE AGREGACION) | postgres | 2.173     | 0.274       | 1.906   | 2.54    | 1.979       | 5           |
| Query 1 (BASE AGREGACION) | spark    | 0.885     | 0.48        | 0.542   | 1.796   | 0.581       | 5           |
| Query 2 (BASE COMPLEX)    | postgres | 2.423     | 1.48        | 1.555   | 5.375   | 1.719       | 5           |
| Query 2 (BASE COMPLEX)    | spark    | 3.039     | 0.699       | 2.455   | 4.344   | 2.666       | 5           |
| Query 1 GPT-5 (PG)        | postgres | 1.13      | 0.458       | 0.536   | 1.65    | 1.361       | 5           |
| Query 2 GPT-5 (PG)        | postgres | 0.962     | 0.17        | 0.85    | 1.289   | 0.853       | 5           |
| Query 1 GEMINI (PG)       | postgres | 0.528     | 0.02        | 0.518   | 0.568   | 0.519       | 5           |
| Query 2 GEMINI (PG)       | postgres | 0.919     | 0.097       | 0.853   | 1.104   | 0.858       | 5           |
| Query 1 CLAUDE (PG)       | postgres | 0.528     | 0.02        | 0.517   | 0.569   | 0.519       | 5           |
| Query 2 CLAUDE (PG)       | postgres | 1.072     | 0.017       | 1.056   | 1.106   | 1.066       | 5           |
| Query 1 GPT-5 (Spark)     | spark    | 0.729     | 0.228       | 0.515   | 1.155   | 0.66        | 5           |
| Query 2 GPT-5 (Spark)     | spark    | 3.134     | 0.309       | 2.8     | 3.584   | 2.968       | 5           |
| Query 1 GEMINI (Spark)    | spark    | 0.619     | 0.095       | 0.553   | 0.799   | 0.555       | 5           |
| Query 2 GEMINI (Spark)    | spark    | 2.499     | 0.196       | 2.341   | 2.876   | 2.424       | 5           |
| Query 1 CLAUDE (Spark)    | spark    | 0.561     | 0.149       | 0.409   | 0.807   | 0.531       | 5           |
| Query 2 CLAUDE (Spark)    | spark    | 1.455     | 0.284       | 1.093   | 1.898   | 1.322       | 5           |

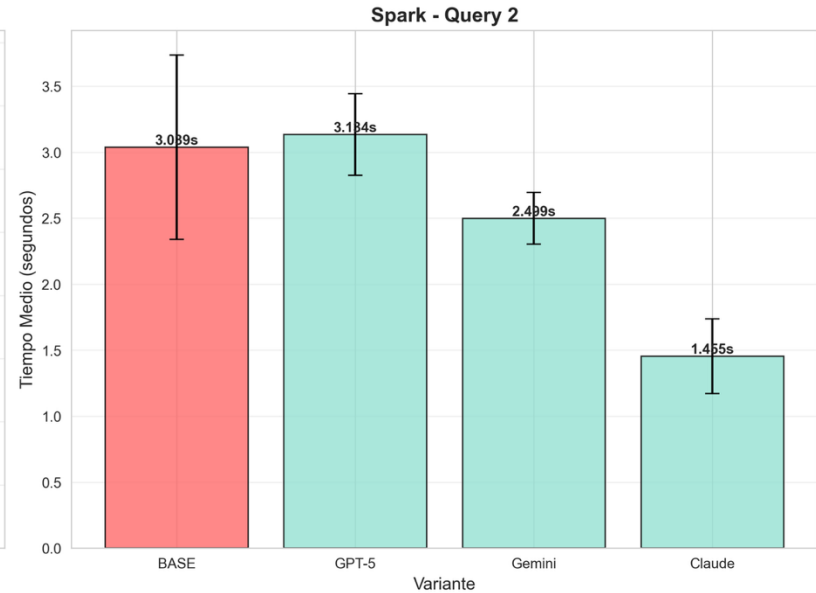
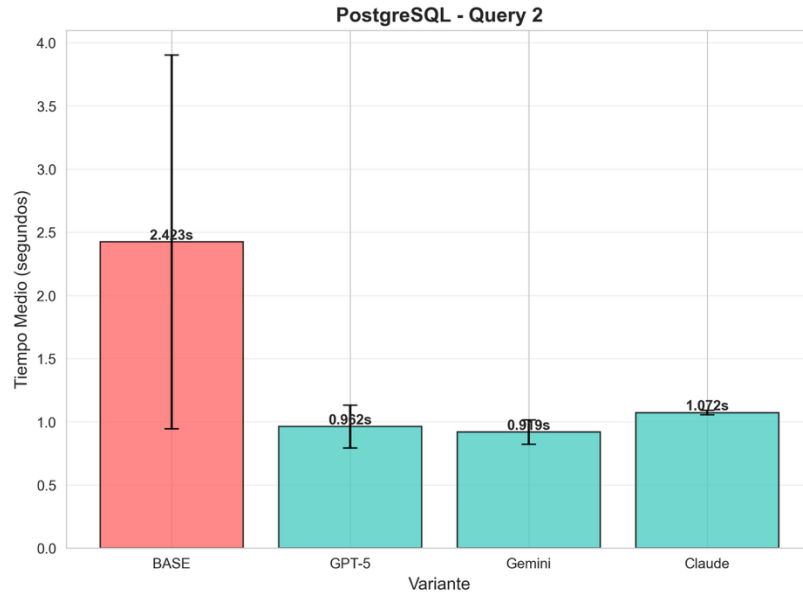




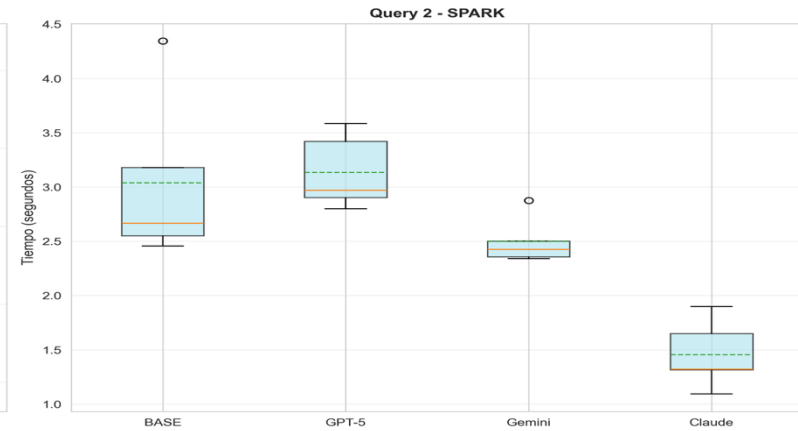
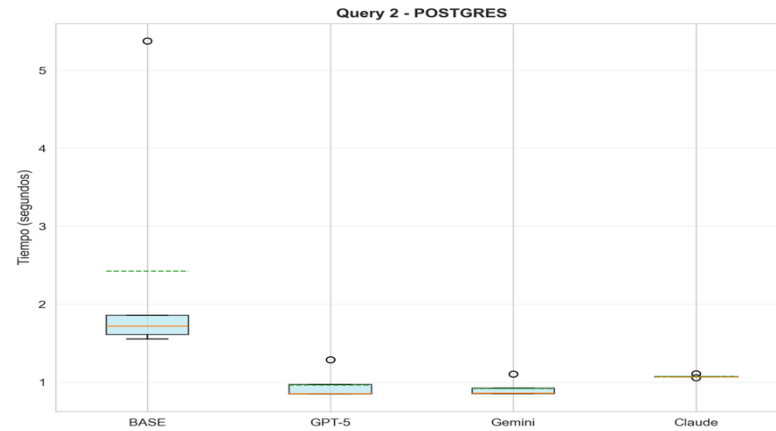
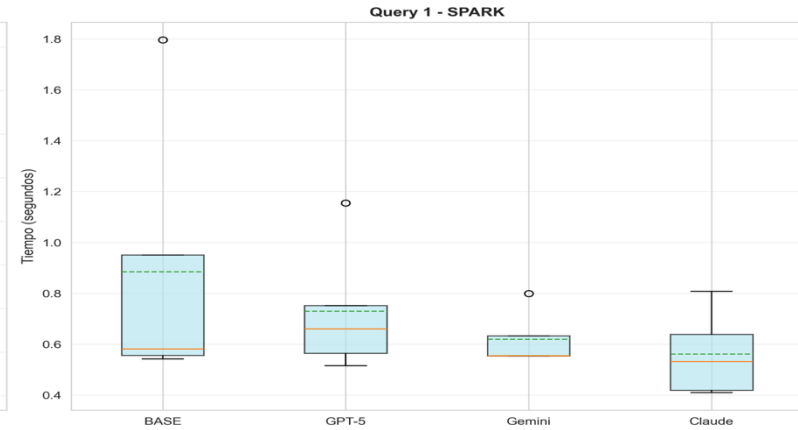
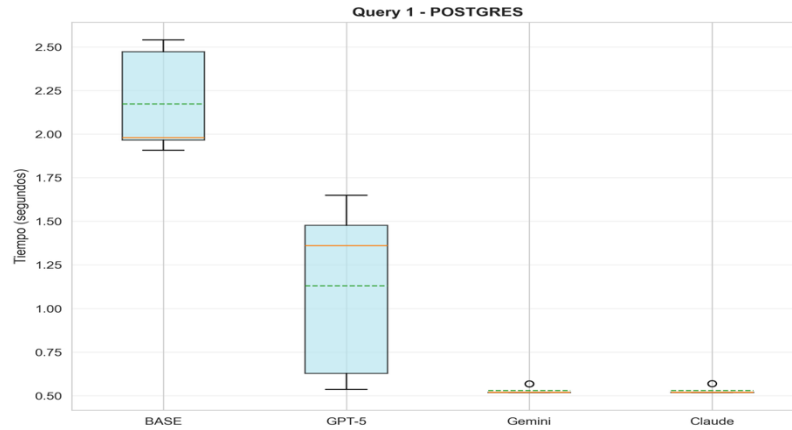
# Comparacion Query 1



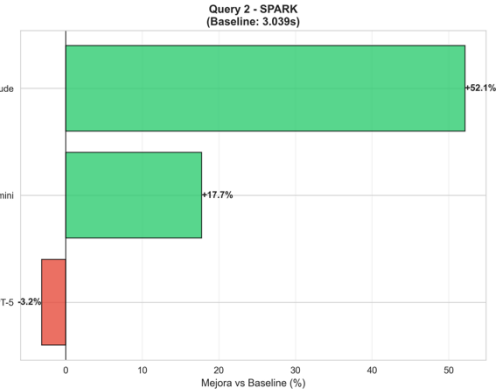
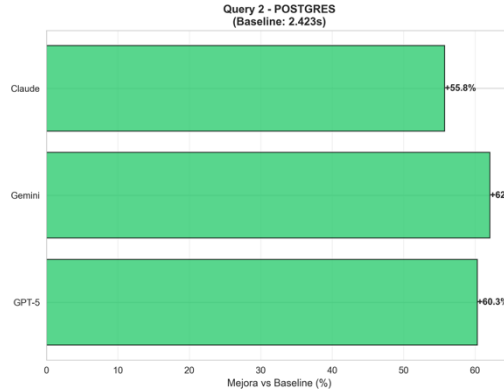
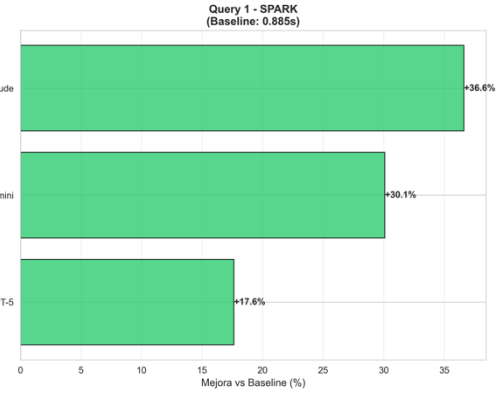
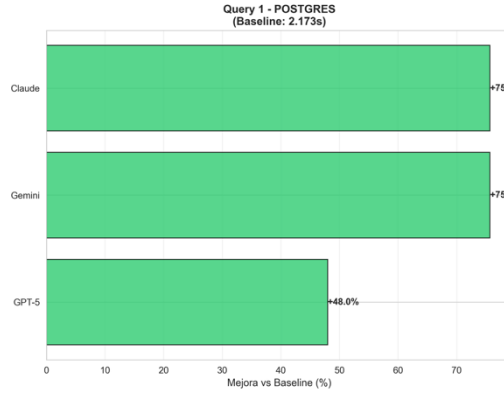
# Comparacion Query 2



# Resultados y Análisis



# Resultados y Análisis



# Resultados y Análisis



# FIN – Qué IA lo hace mejor?

Gabriel Cárdbaba López

